Servlet Demo Document (Create Case)
Customer Registration

Problem Statement

Develop a web application using MVC approach which provides below features:

- Register a customer

Below details of customer have to be captured and a unique customer ID should be displayed to the
customer.

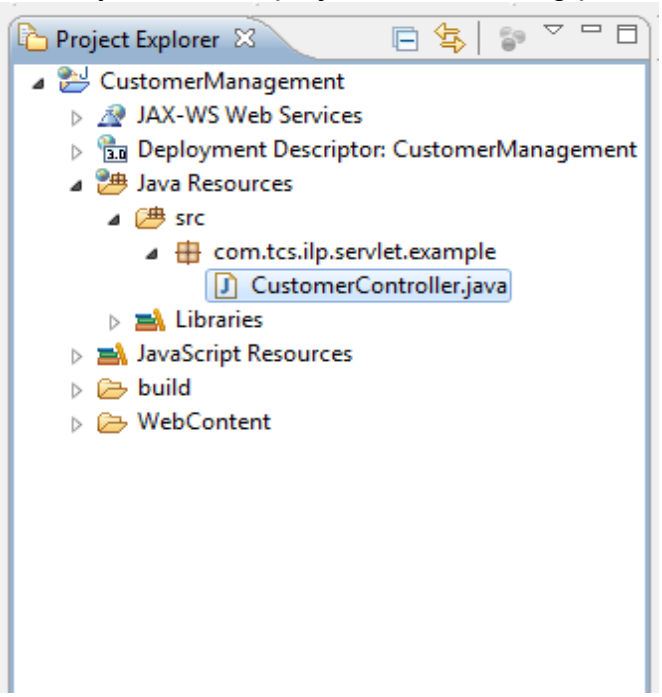
- Name
- Age
- Country
- Address
- Gender
- Areas Of Interest

Use Java POJOs as model, Servlet as controller and HTML for developing view component.

1.Application development

1.1 Design of package structure

Create dynamic web project with following package structure



1.2 Design of view layer(html /jsp page)

Capture all above mentioned fields in the registerCutomer.html .Place registerCustomer.html in the webcontent folder.This act as the view layer .Map this as the welcome file in the web.xml(section 4)

registerCustomer.html

```
1<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
2<html>
3<head>
4<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
5<title>Customer  Details</title>
6<script src="AddDetails.js">
7
8</script>
9</head>
10<body>
11<form method="post" action="CustomerController">
12<h4>Customer Details</h4>
13<table>
14    <tr>
15        <td>Name:</td>
16        <td><input type="text" name="Name" id="name"></input></td>
17    </tr>
18    <tr>
19        <td>Age:</td>
20        <td><input type="text" name="Age" id="age"></input></td>
21    </tr>
22    <tr>
23        <td>Country:</td>
24        <td><select name="country">
25            <option value="India">India</option>
26            <option value="US">US</option>
27            <option value="Russia">Russia</option>
28            <option value="China">China</option>
29        </select></td>
30    </tr>
```

```
        <td>Address:</td>
        <td><textarea rows="2" cols="20" name="address">
</textarea></td>
    </tr>
    <tr>
        <td>Gender:</td>
        <td><input type="radio" name="Gender" value="M">Male</td>
    </tr>
    <tr>
        <td> </td>
        <td><input type="radio" name="Gender" value="F">Female</td>
    </tr>

    <tr>
        <td>Areas of Interest</td>
        <td><input type="checkbox" name="areaofinterest" value="IT">IT</td>
    </tr>
    <tr>
        <td> </td>
        <td><input type="checkbox" name="areaofinterest" value="marketing">Marketing
        </td>
    </tr>
    <tr>
        <td> </td>
        <td><input type="checkbox" name="areaofinterest" value="sales">Sales
        </td>
    </tr>

    <tr>
        <td> </td>
        <td><input type="submit" value="Register"></input></td>
```

2.2 Design of the controller(servlet)

Create a CustomerController which act as a layer between the view i.e registerCustomer.html and the database here in our case oracle databse .Code snippet is given below.

```java
//import com.tcs.ilp.servlet.dao.DBConnection;

public class CustomerController extends HttpServlet {
    private PreparedStatement preparedStatement1 = null;
    private PreparedStatement preparedStatement2 = null;
    private ResultSet rs = null;
    String customerName=null;
    String age =null;
    String country = null;
    String address = null;
    String gender = null;
    String[] areaOfInterest ;
    protected void doPost(HttpServletRequest request,
            HttpServletResponse response) throws ServletException, IOException {
            Connection con=null;
            String interest = "";

        try {
        //  String interest = "";


            // Reads all content from request object
             customerName = request.getParameter("name");
             age = request.getParameter("age");
             country = request.getParameter("country");
             address = request.getParameter("address");
             gender = request.getParameter("Gender");
             areaOfInterest = request
                    .getParameterValues("areaofinterest");
            for (String m : areaOfInterest) {
                interest = interest + m + ",";
            }
        }


            catch (Exception e) {
                System.out.println(e);
```

Using request.getParameter and request.getparameterValues we get the values entered in the registerCustomer.html

In the below code we create connection to the database and insert data to the data base with help of Jdbc APIs .After inserting the data the customerId is displayed back to the customer

```
        }
        //Creating the connections

        try
        {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            con=DriverManager.getConnection("jdbc:oracle:thin:@172.24.137.30:1521:orcl",
                    "e308369","fiugNHtqz");

            int custId = 0;

            // insert the details using bean object and prepared statement
            preparedStatement1 = con
                    .prepareStatement("insert into tbl_customer values(customer_sequence.nextval,?,?,?,?,?,?)");
            preparedStatement1.setString(1, customerName);
            preparedStatement1.setString(2, age);
            preparedStatement1.setString(3, country);
            preparedStatement1.setString(4, address);
            preparedStatement1.setString(5, gender);
            preparedStatement1.setString(6,interest);
            preparedStatement1.executeUpdate();

            // Retrieves the generated customer ID from dual
            preparedStatement2 = con
                    .prepareStatement("select customer_sequence.currval from dual");
            rs = preparedStatement2.executeQuery();
            while (rs.next()) {
                // Customer cust=new Customer();
                custId = rs.getInt(1);
            }
            PrintWriter out = response.getWriter();
            out.println("The generated customer ID is " + custId);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
```

When we run this application we will get a customer registration page

http://localhost:8080/CustomerManagement/registerCustomer.html

Customer Details

Name: Sarannya
Age: 23
Country: India
Address: Nehru Street
Gender: Male
Female
Areas of Interest IT
Marketing
Sales
Register

On click of register button next page with id is generated

http://localhost:8080/CustomerManagement/CustomerController

The generated customer ID is 8

2.Limitations to the above design

1.Lack of Modularity:
Whole code is written in the servlet class .If we want to go for any enhancements its error prone and cumbersome. We many not be able to visualize the difficulty level with

the above mentioned a application.But consider an application which has around 10 registration pages (say customer,product etc) which make use of four databases(say sybase,db2,oracle,mysql) .If we want to make modification to the connection expression of say oracle database ,we have to go through the entire code and make nessassary changes which is cumbersome and error prone.In short any change to the code is an error prone process

Steps to modularize code
1.Redesign the package structure to include different package for controller,databaseconnection ,dao ,service etc which will enhance the maintainability of the code

2.Violation of OOPs concepts:
Data from the registerCustomer is captured in the servlet and the data as such is passed on to the next database layer which is a violation of OOPs concepts

To adhere to the OOPs concepts we create the bean class Customer.java ,so that data along with its method is wrapped together and is passed from one layer as object and not as data

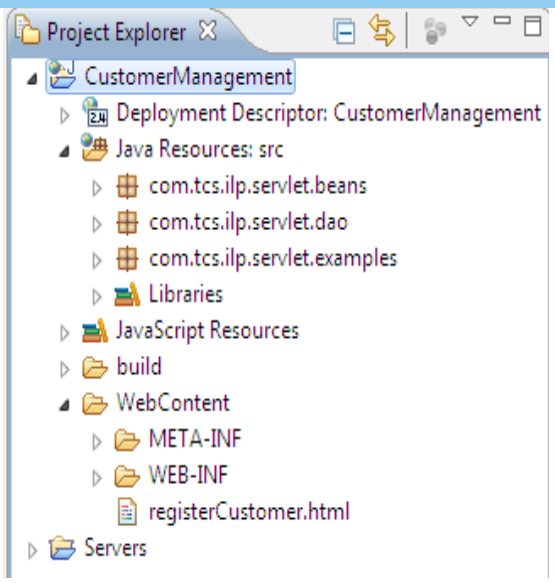
NOW LETS REDESIGN THE ENTIRE SYSTEM KEEPING THESE POINTS IN MIND

3. Redesign of CustomerManagement application

1.Create dynamic web project as shown in section 2.1(Package Structure)

3.1 Redesign of package structure

Create dynamic Web project **CustomerManagement** with the following structure



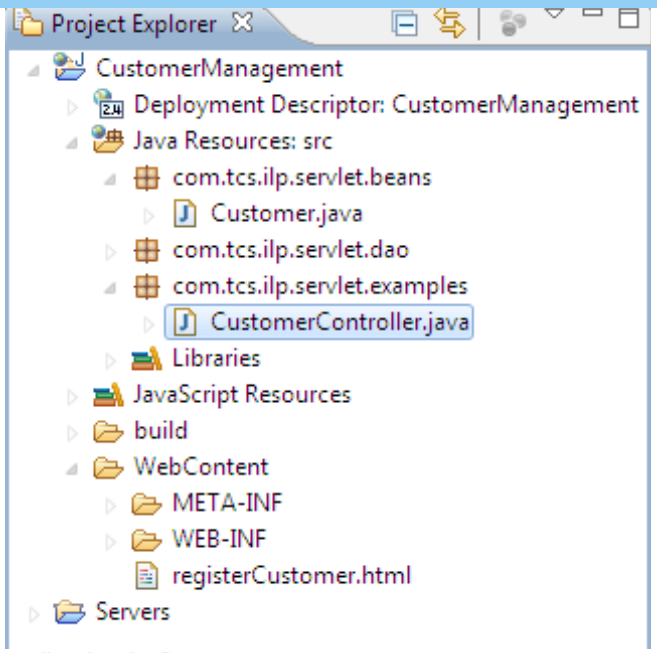
Place the registerCustomer.html we created earlier in the WebContent.This act as the welcome file .We need to map this web.xml

3.2 Redesign of controller

Now the servlet , ie CustomerController will read content from request object(Using request.getParameter()) and forms bean object(Customer refer section3.3),creates DAO object (refer section 3.5)and passes within.

Now the servlet handles only request and response related activities and rest of the activities are handled by respective classes

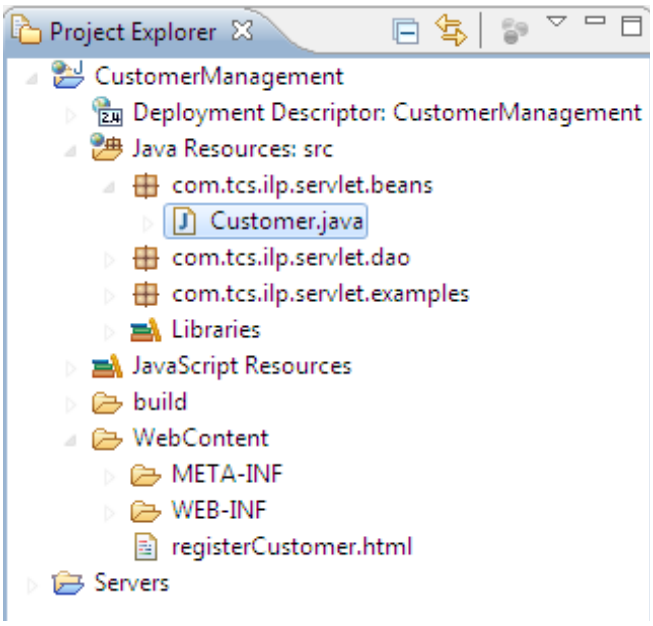
As mentioned in section 2.2 controller and the welcome file needs to be mapped in the welcome file




```
41    protected void doPost(HttpServletRequest request,
42            HttpServletResponse response) throws ServletException, IOException {
43        try {
44            String interest = "";
45            Customer customer = new Customer();
46            CustomerService customerservice=new CustomerService();
47            // Reads all content from request object
48            String customerName = request.getParameter("name");
49            String age = request.getParameter("age");
50            String country = request.getParameter("country");
51            String address = request.getParameter("address");
52            String gender = request.getParameter("Gender");
53            String[] areaOfInterest = request
54                    .getParameterValues("areaofinterest");
55            for (String m : areaOfInterest) {
56                interest = interest + m + ",";
57            }
58            // Forms bean object with the request object.
59            customer.setCustomerName(customerName);
60            customer.setAge(age);
61            customer.setCountry(country);
62            customer.setAddress(address);
63            customer.setGender(gender);
64            customer.setAreasOfInterest(interest);
65            int custId=customerservice.createCustomer(customer);
66            // Printwriter to send the response back to the user as generated
67            // customer id.
68            PrintWriter out = response.getWriter();
69            out.println("The generated customer ID is " + custId);
70        } catch (Exception e) {
71            System.out.println(e);
```

3.3 Bean class

Create a bean class Customer inside the package com.tcs.ilp.servlet.beans to hold the getters and setters of customer details.By doing this we can overcome viotaion to oops concepts that we discussed in section 2



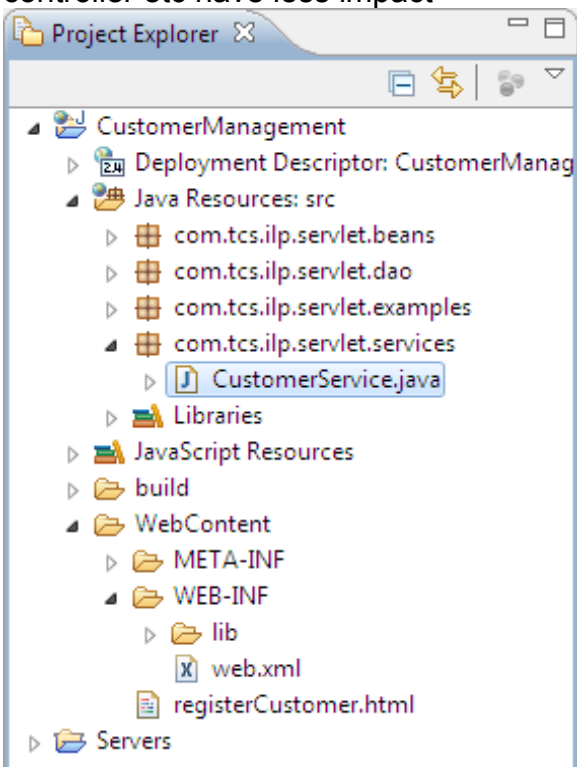
Customer.java

```java
package com.tcs.ilp.servlet.beans;

public class Customer {
    private String customerID;
    private String customerName;
    private String age;
    private String country;
    private String address;
    private String gender;
    private String areasOfInterest;

    public String getCustomerID() {
        return customerID;
    }

    public void setCustomerID(String customerID) {
        this.customerID = customerID;
    }

    public String getCustomerName() {
        return customerName;
    }

    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public String getAge() {
        return age;
    }
```

```java
    public String getCountry() {
        return country;
    }

    public void setCountry(String country) {
        this.country = country;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public String getGender() {
        return gender;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }

    public String getAreasOfInterest() {
        return areasOfInterest;
    }

    public void setAreasOfInterest(String areasOfInterest) {
        this.areasOfInterest = areasOfInterest;
    }
```

3.3 Service Class

This act as a layer between the controller and dao .This layer is introduced to reduce the dependency say if we plan to migrate from oracle to sybase there can be changes say for example query formation etc. By introducing service layer other layers like controller etc have less impact



CustomerService.java

```java
package com.tcs.ilp.servlet.services;

import com.tcs.ilp.servlet.beans.Customer;
import com.tcs.ilp.servlet.dao.CustomerDAO;

public class CustomerService {

    public int createCustomer(Customer customer) throws Exception
    {
    CustomerDAO customerdao = new CustomerDAO();
    return customerdao.addCustomerDetails(customer);
    }

}
```

3.4 Connection class

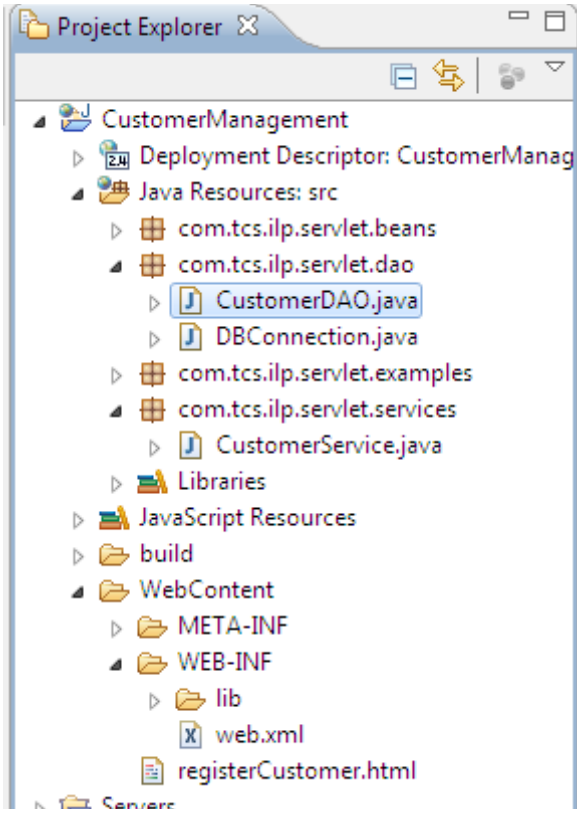
This will handle every thing related to connection say oracle driver details to get and close connection etc

DBConnection.java

```java
package com.tcs.ilp.servlet.dao;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    public static Connection getConnection() throws SQLException{
        Connection con=null;
        try
        {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            con=DriverManager.getConnection("jdbc:oracle:thin:@172.24.137.30:1521:orcl",
                    "e111111","yourpassword");
        }
        catch(ClassNotFoundException e)
        {
            e.printStackTrace();
        }
        return con;
    }

    public static void closeConnection(Connection con)
   {
       if(con!=null)
       {
           try{
               con.close();
           }catch(SQLException e){
           e.printStackTrace();
           }
```

3.5 DAO Class

All Dml and DRLs are handlled in this class .Here in customer management insert statement is done using prepared statement

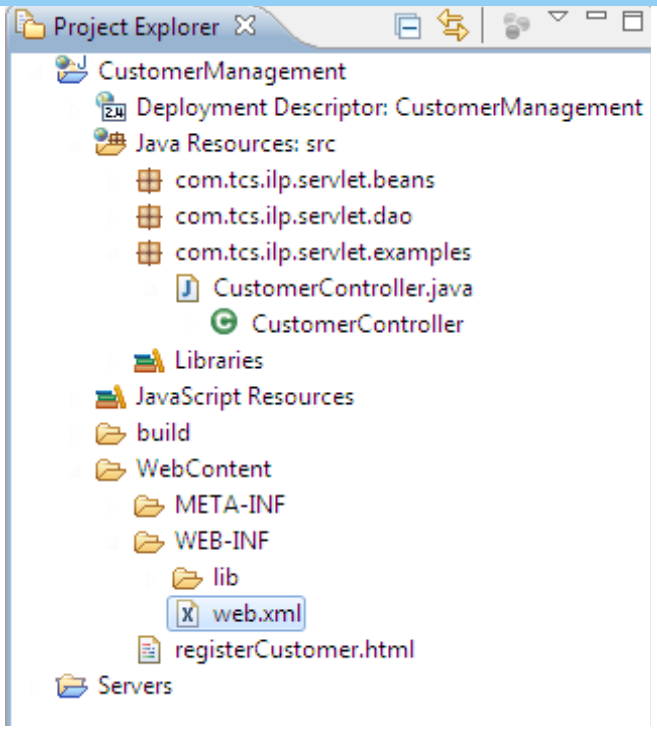


CustomerDAO.java

```java
package com.tcs.ilp.servlet.dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import com.tcs.ilp.servlet.beans.*;

public class CustomerDAO {
    private Connection connect = null;
    private PreparedStatement preparedStatement1 = null;
    private PreparedStatement preparedStatement2 = null;
    private ResultSet rs = null;

    public int addCustomerDetails(Customer customer) throws Exception {
        int custId = 0;
        connect = DBConnection.getConnection();
        // insert the details using bean object and prepared statement
        preparedStatement1 = connect
                .prepareStatement("insert into tbl_customer values(customer_sequence.nextval,?,?,?,?,?,?)");
        preparedStatement1.setString(1, customer.getCustomerName());
        preparedStatement1.setString(2, customer.getAge());
        preparedStatement1.setString(3, customer.getCountry());
        preparedStatement1.setString(4, customer.getAddress());
        preparedStatement1.setString(5, customer.getGender());
        preparedStatement1.setString(6, customer.getAreasOfInterest());
        preparedStatement1.executeUpdate();

        // Retrieves the generated customer ID from dual
        preparedStatement2 = connect
                .prepareStatement("select customer_sequence.currval from dual");
        rs = preparedStatement2.executeQuery();
        while (rs.next()) {
            // Customer cust=new Customer();
            custId = rs.getInt(1);
        }
        DBConnection.closeConnection(connect);
        return custId;
    }

}
```

4) web.xml

Required mapping for servlet and html shoul be done in web.xml


Project Explorer
CustomerManagement
Deployment Descriptor: CustomerManagement
Java Resources: src
com.tcs.ilp.servlet.beans
com.tcs.ilp.servlet.dao
com.tcs.ilp.servlet.examples
CustomerController.java
CustomerController
Libraries
JavaScript Resources
build
WebContent
META-INF
WEB-INF
lib
web.xml
registerCustomer.html
Servers

web.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app id="WebApp_ID" version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001
    <display-name>CustomerManagement</display-name>
    <servlet>
        <description>
        </description>
        <display-name>CustomerController</display-name>
        <servlet-name>CustomerController</servlet-name>
        <servlet-class>
        com.tcs.ilp.servlet.examples.CustomerController</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>CustomerController</servlet-name>
        <url-pattern>/CustomerController</url-pattern>
    </servlet-mapping>
    <welcome-file-list>
        <welcome-file>registerCustomer.html</welcome-file>
        <welcome-file>index.htm</welcome-file>
        <welcome-file>index.jsp</welcome-file>
        <welcome-file>default.html</welcome-file>
        <welcome-file>default.htm</welcome-file>
        <welcome-file>default.jsp</welcome-file>
    </welcome-file-list>
</web-app>
```